

DBMS Lab 项目说明

项目概览

这个项目使用 Flex + Bison + C 实现了一个课程实验用的小型 DBMS 原型，运行环境面向 WSL / Linux。

项目目标分为两层：

1. 完成上机题目要求的 11 类基本 SQL 语句及相关功能。
2. 在此基础上，尽量实现改进建议中的四个方向：
 - 安全
 - 视图
 - 索引 / 主码 / 外码 / 约束
 - 事务 / 日志

当前版本已经可以稳定编译，并且可以完整跑通 [test.sql](#) 中的基础功能验收脚本。

目录结构与文件作用

根目录

- [Makefile](#)
负责编译、清理和基础运行命令。
- [README.md](#)
当前这份项目总说明。
- [验收指导.md](#)
从头编译、测试、调试、验收的操作说明。
- [实验规划.md](#)
基础版本的设计与实现规划文档。
- [实验规划_改进.md](#)
加入改进方向后的设计规划文档。
- [操作.md](#)
基础版本的操作说明。
- [操作_改进.md](#)
改进版本的操作说明。

- [.gitignore](#)

忽略编译生成物和运行时数据目录内容。

src/

- [dbms.c](#)

程序入口。

负责：

- 初始化 data/ 根目录和 data/sys.dat
- 解析命令行参数
- 支持 ./dbms test/test.sql
- 支持 ./dbms --no-auth test/test.sql
- 未提供脚本路径时从标准输入读取 SQL

- [dbms.h](#)

全局数据结构定义。

包括：

- 字段定义 FieldDef
- 约束定义 ConstraintDef
- CREATE / INSERT / SELECT / DELETE / UPDATE 的语法树结构
- 视图、索引、事务日志、授权相关结构

- [sql.l](#)

Flex 词法规则。

负责：

- 关键字识别
- 标识符、整数字面量、字符串字面量识别
- 支持大小写不敏感关键字
- 支持 // 与 -- 注释

- [sql.y](#)

Bison 语法规则与语义动作。

负责：

- 将 SQL 解析为语法树
- 在语义动作中调用执行器函数
- 提供 error ';' 错误恢复

- [executor.c](#)

核心执行层。

负责：

- 数据库、表、记录的增删改查
- 多表查询

- 条件计算
- 安全与授权
- 视图、索引、约束、事务日志的主要行为

- [storage.c](#)

存储层辅助函数。

负责：

- 加载表结构
- 加载表数据
- 保存表数据
- 约束元数据加载
- 行、条件相关基础工具

- [storage.h](#)

存储层函数声明。

test/

- [test.sql](#)

课程实验的主测试脚本。

覆盖：

- 建库 / 删库 / 显示库 / 选库
- 建表 / 删表 / 显示表
- 插入
- 单表查询
- 多表查询
- 删除
- 更新
- 错误恢复
- 小写关键字
- 缺列插入
- 多层括号条件

data/

- [data/.gitkeep](#)

保留空目录用。

- 运行时会自动生成：

- data/sys.dat
- data/<dbname>/sys.dat

- data/<dbname>/*.dat
- data/<dbname>/views.meta
- data/<dbname>/constraints.meta
- data/<dbname>/grants.meta
- data/<dbname>/users.meta
- data/<dbname>/indexes/
- data/<dbname>/logs/

当前系统基本功能实现情况

已稳定实现的基础 SQL 功能

当前版本已经可以编译并跑通基础实验脚本，以下 11 类基础功能可用：

1. CREATE DATABASE
2. DROP DATABASE
3. SHOW DATABASES
4. USE DATABASE
5. CREATE TABLE
6. DROP TABLE
7. SHOW TABLES
8. INSERT
9. SELECT
10. DELETE
11. UPDATE

已验证通过的基础行为

- 重复建库会报错
- 错误语法行会被跳过并继续执行后续语句
- 小写关键字可以正常识别
- 同一行多条语句可以连续执行
- 缺列插入会补默认值
 - INT 默认值为 0
 - CHAR 默认值为空串
- WHERE (((...))) 这类多层括号条件可正确解析
- 多表查询可执行简单笛卡尔积与条件过滤

最近一次基础回归结果

在干净状态下执行：

```
make clean && make
rm -rf data
mkdir -p data
touch data/.gitkeep
./dbms test/test.sql
```

结果：

- 编译成功
- 脚本完整执行到末尾
- 没有崩溃、段错误、非法指针错误
- 输出结果与当前实验预期基本一致

改进建议四个方向实现情况

1. 安全

当前状态：**部分实现**

已实现：

- CREATE USER
- DROP USER
- LOGIN
- GRANT
- REVOKE
- 表级权限检查：
 - SELECT
 - INSERT
 - UPDATE
 - DELETE

当前特点：

- 默认用户是 ADMIN
- ADMIN 不受普通授权限制

- 普通用户若没有被授权，会在访问表时被拒绝

当前不足：

- 没有密码机制
- 没有角色系统
- `--no-auth` 当前只被识别，但没有真正关闭鉴权开关

2. 视图

当前状态：**基本可用，但不是完整数据库级实现**

已实现：

- `CREATE VIEW`
- `DROP VIEW`
- `SHOW VIEWS`
- `SELECT * FROM view`

当前特点：

- 视图定义保存在 `views.meta`
- 当前版本已经修复了“查询视图导致崩溃”的问题

当前不足：

- 更适合 `SELECT * FROM 视图名`
- 不建议依赖复杂的“在视图上再投影、再加 WHERE、再嵌套视图”场景
- 视图查询目前通过子进程执行定义 SQL，属于实用型修补方案，不是完整查询重写架构

3. 索引 / 主码 / 外码 / 约束

当前状态：**部分实现**

已实现：

- `CREATE INDEX`
- `DROP INDEX`
- `PRIMARY KEY`
- `UNIQUE`
- `NOT NULL`
- `FOREIGN KEY ... REFERENCES ...`

已验证行为：

- 主码重复插入会被拒绝
- 唯一值重复插入会被拒绝
- 外码引用不存在的父表值会被拒绝
- 删除被外码引用的父表行时会阻止删除

当前不足：

- 索引优化只覆盖较窄的查询场景
- 不属于通用查询优化器
- 索引主要用于单表、等值、简单条件查询

4. 事务 / 日志

当前状态：**基础实现已可用**

已实现：

- BEGIN
- COMMIT
- ROLLBACK
- 事务日志文件写入
- 事务内读到未提交修改

已修复问题：

- 之前 COMMIT 后更新不会真正落盘
- 当前版本已经修复，提交后数据会保留

当前不足：

- 事务实现是课程实验级别，不是完整数据库事务系统
- 没有并发控制
- 没有崩溃恢复
- 没有 checkpoint
- 没有 WAL 严格语义保证

当前仍然重要的待改进部分

下面这些不是当前基础验收阻断项，但如果你继续完善项目，它们是优先级较高的方向：

1. `--no-auth` 语义应真正生效
当前只是识别参数，没有把 `auth_enabled` 关闭。
2. 视图执行架构仍可继续正规化
当前方案已经稳定，但仍偏“工程补丁式”实现。
3. 索引能力仍比较窄
适合课程展示，不适合作为完整索引子系统理解。
4. 事务仍缺真正恢复能力
现在更接近“单会话日志 + 回滚/提交”而非完整恢复系统。
5. 输出格式仍是实验风格
适合验收，不适合进一步扩展成更通用的人机交互界面。

环境依赖

建议在 Ubuntu / WSL 中运行。

需要工具：

- `gcc`
- `make`
- `flex`
- `bison`

安装命令：

```
sudo apt update
sudo apt install -y gcc make flex bison
```

常用命令

编译

```
make
```

清理编译产物

```
make clean
```


清空运行数据

```
make reset
```

运行主测试脚本

```
./dbms test/test.sql
```

从标准输入执行

```
./dbms
```

带可选参数执行

```
./dbms --no-auth test/test.sql
```

推荐阅读顺序

如果你要快速理解整个项目，推荐按这个顺序读：

1. [README.md](#)
2. [验收指导.md](#)
3. [dbms.c](#)
4. [sql.l](#)
5. [sql.y](#)
6. [executor.c](#)
7. [storage.c](#)